



Goss IP

GOSS IP

Videovigilancia que avisa a tiempo

Memoria técnica del prototipo

Reto InnovaTIC's UAEMéx 2026 · Fase 2

Versión del 7 de octubre de 2026 (Bootcamp, día 1)

Integrante	Rol en el equipo
Persona asesora	

Repositorio: github.com/Michou001/Sistema-videovigilancia · rama innovatics-final

1. Problema y alcance

En la mayoría de los sitios con videovigilancia, las cámaras graban pero casi nadie las está mirando en el momento en que pasa algo. Un monitorista vigila muchas pantallas a la vez y lo importante ocurre en una mientras mira otra. Por eso el video suele usarse después, para reconstruir lo que pasó, y no para reaccionar. Encontrar una placa o un momento en días de grabación se hace a mano.

GOSS IP analiza el video de cámaras IP en tiempo real y convierte lo que importa en alertas que el monitorista puede atender: una placa en lista negra, una persona caída, alguien en una zona fuera de horario, una cámara tapada o sin señal. Cada alerta llega con folio, foto, clip de los segundos antes y después, y su ubicación en el mapa.

Alcance del prototipo

- Lectura de **placas mexicanas** y cruce contra una lista negra con vigencia.
- Detección de **personas en lista negra** por rostro, solo con fundamento legal declarado.
- **Movimiento súbito, caída, manos arriba y posible agresión** por análisis de pose.
- **Zonas y reglas con horario**: intrusión, cruce de línea con sentido, merodeo y conteo.
- **La cámara misma**: lente tapado, cámara movida, pérdida de video y cámara caída.
- **Plataforma web** para el monitorista: monitoreo en vivo, registro, mapa, catálogo de cámaras, lista negra, usuarios y bitácora.
- **Avisos** al celular (Telegram, correo, WhatsApp o webhook) para las alertas críticas.
- **Revisión de lecturas**: el operador confirma una placa o la marca como "no es placa"; lo revisado alimenta el reentrenamiento del OCR.

Fuera del alcance actual: detección de armas (apagada: no tiene un modelo confiable, ver sección 12) e integración directa con centros de monitoreo externos (prevista por webhook).

2. Usuarios y beneficiarios

Quién	Qué hace en GOSS IP
Monitorista (usuario principal)	Recibe alertas con foto, clip y ubicación; las atiende con folio y nota; busca en el registro.
Responsable de seguridad	Administra lista negra, reglas por horario y usuarios; revisa reportes y la bitácora de auditoría.
Instalador / área de TI	Da de alta cámaras desde el navegador y sabe qué puede hacer cada una antes de instalarla.
Comunidad (beneficiarios)	Estudiantes, personal y visitantes: respuesta más rápida, con su privacidad cuidada desde el diseño.

3. Propuesta de valor e innovación

GOSS IP no sustituye al monitorista: le quita la tarea imposible de mirar todo y le deja la de decidir. Lo que lo distingue de un sistema de grabación tradicional o de la analítica básica de una cámara:

- **Placas mexicanas de verdad.** Conoce los formatos de la NOM-001-SCT-2-2016 y anteriores: sabe qué posición es letra y cuál dígito, así que corrige errores típicos del OCR (A8C-I23 → ABC-123) y dice el tipo de placa y la entidad.
- **Corre en la red local.** El video no sale del sitio. Si la red o el servidor se caen, el borde guarda los eventos en disco y los reenvía al volver.
- **Funciona con cámaras que ya existen:** RTSP, ONVIF y los eventos propios de Hikvision (ISAPI).
- **Privacidad desde el diseño:** el rostro de quien no está en la lista negra nunca se guarda.
- **Avisa mientras pasa:** una persona de la lista negra se reporta en cuanto se le ve de frente, con ella todavía frente a la cámara; no hay que esperar a que se vaya.
- **Reglas explicables:** cada alerta dice por qué salió; una coincidencia dudosa se presenta como "posible", nunca como hecho.
- **Revisión del pasado:** al dar de alta una placa, el sistema busca si ya pasó frente a una cámara.
- **Catálogo de cámaras:** encuentra, identifica y recomienda la instalación de cada cámara, sin afirmar especificaciones que la cámara no reportó.

4. Arquitectura



Tres procesos con responsabilidades separadas, unidos por un contrato de evento:

- **Worker de borde** (Python, GPU). Lee el video por RTSP, corre los detectores y sigue a cada objeto. No decide qué es una alerta: reporta "vi esto, con esta confianza".
- **API** (FastAPI). Cruza cada evento contra la lista negra y las reglas por horario, decide la severidad, guarda en la base de datos y avisa. Como la decisión está aquí, la lista negra y las reglas se cambian sin tocar los workers.
- **Dashboard web.** Una sola página sin framework ni CDN: funciona en una red sin internet. Recibe eventos y alertas por WebSocket y el video por WebRTC (go2rtc) o MJPEG. El video en vivo (20

cuadros por segundo) va aparte del análisis: las cajas se dibujan en el navegador y el video nunca espera a la IA.

Recorrido de una placa en lista negra

1. La cámara entrega 20 cuadros por segundo; el worker analiza 8 (suficiente y deja GPU libre).
2. El detector encuentra la placa (confianza mínima 0.60); el seguimiento la asocia a un mismo vehículo y se hacen hasta 10 lecturas de OCR mientras está en cuadro.
3. Solo se reporta si al menos **3 lecturas independientes** coinciden con confianza alta (0.75): así una etiqueta o un letrero no se convierte en placa. Se elige la lectura por consenso carácter por carácter, se corrige por formato mexicano y se pide una foto del canal principal (3200 × 1800) para la evidencia.
4. La API la cruza contra la lista negra: idéntica o corregida → crítica; a una letra de distancia → aviso de "posible" coincidencia.
5. El dashboard muestra el banner y suena; la alerta tiene folio (ALR-000123), foto y clip; si está configurado, llega el aviso al celular. El operador la atiende y deja nota en la bitácora.

5. Tecnología

Componente	Tecnología	Licencia
Detección de placas	YOLOv9 (open-image-models), ONNX Runtime GPU	MIT
OCR de placas	fast-plate-ocr (modelo entrenado con placas)	MIT
Personas y vehículos, pose	YOLO11 y YOLO11-pose (Ultralytics), PyTorch	AGPL-3.0
Rostros	InsightFace buffalo_l (ArcFace, 512 dimensiones)	Pesos: no comercial
Seguimiento	ByteTrack	MIT
API	FastAPI, SQLAlchemy, Alembic	MIT
Base de datos	SQLite (1-2 cámaras) o PostgreSQL + Redis	Libres
Video en el navegador	go2rtc (WebRTC) o MJPEG	MIT
Dashboard	HTML + JavaScript sin framework, Leaflet, Lucide	BSD-2 / ISC
Despliegue	Windows o Linux; Docker con HTTPS (Caddy)	Apache-2.0

Detalle y alternativas para uso comercial en docs/licencias.md del repositorio.

6. Módulos y estado real

Módulo	Estado	Evidencia o pendiente
Placas mexicanas	Funciona	35/35 lecturas en fotos de prueba de lado, desde arriba, lejos y con poca luz (antes 20/35). Ver ADR-001. Exige 3 lecturas coincidentes; el operador puede marcar "no es placa".
Lista negra con vigencia	Funciona	Revisión del pasado al dar de alta; coincidencia difusa a 1 carácter.
Rostros (lista negra)	Funciona	Evento ~1 s después de ver el rostro de frente, con la persona aún en cuadro. Umbral 0.50 de fábrica: falta calibrarlo con

		personas reales y su consentimiento.
Movimiento súbito	Funciona	Sin falsos positivos en actividad normal; falta probar con un forcejeo real.
Caída, manos arriba, agresión	Validar en campo	Probado con el modelo real; falta un caso real.
Zonas y reglas con horario	Funciona	Editor de zonas en el dashboard; la API juzga con la hora del evento.
Cámara sabotada o caída	Funciona	Caída detectada y recuperada en 30 s (registro real).
Clips y avisos	Funciona	Clip de 10 s antes y después; Telegram, correo, WhatsApp, webhook. Telegram configurado y probado con el bot del equipo (7 oct).
Búsqueda por descripción	Opcional	"camioneta blanca" en español; probado con el modelo real.
Armas	Apagado	YOLO11-COCO no detectó un cuchillo real en mano. El sistema ya junta datos para entrenar un modelo propio.

7. Catálogo de cámaras (desarrollado en el Bootcamp)

Dar de alta una cámara era trabajo de terminal. El catálogo lo lleva al navegador, en cuatro pasos:

Paso	Qué hace	Detalle
1. Buscar	Encuentra cámaras en la red local	ONVIF WS-Discovery, puertos de cámara (RTSP 554, SDK Hikvision 8000, Dahua 37777...), huella ISAPI y fabricante por prefijo MAC (registro IEEE). No manda contraseñas.
2. Diagnosticar	Identifica la cámara y sus streams	Marca, modelo, firmware y serie por ISAPI u ONVIF; codec, resolución y fps de cada stream; cada ruta se confirma con RTSP DESCRIBE sin abrir el video; vista previa.
3. Recomendar	Qué puede hacer según su montaje	Con altura, distancia, lente y resolución calcula px por metro, px de placa y de rostro, ángulo vertical, alcance y nivel DORI.
4. Dar de alta	Configura y arranca	Escribe la configuración del worker (detectores, eventos ISAPI, foto HD) y lo inicia desde el navegador.

Principios: la contraseña se prueba una sola vez antes de tocar el video (las Hikvision bloquean la IP tras unos cinco intentos fallidos); cada dato muestra su fuente (la cámara, el instalador o una regla de GOSS); los estados son identificada, parcial o manual; y una cámara se reconoce por su serie o MAC aunque el router le cambie la IP.

Reglas del recomendador

Regla	Valor	Origen
Ancho mínimo de placa para leerla	36 px	Medido con el detector y OCR del sistema
Ancho mínimo de rostro	50 px	Umbral del sistema (ArcFace usa recortes de 112 px)
Margen para instalar	× 2	Movimiento, noche y suciedad en sitio

Detección / Observación	25 / 62.5 px/m	IEC 62676-4 (DORI)
Reconocimiento / Identificación	125 / 250 px/m	IEC 62676-4 (DORI)
Ángulo vertical para placas	≤ 30°	Criterio de instalación

8. Seguridad

- **Tres roles** (Consulta, Operador, Administrador). Los permisos los decide la API, no la pantalla.
- **Bitácora de auditoría** de todo cambio y de toda búsqueda por descripción.
- Contraseñas con **bcrypt**; sesión con **revocación** al cambiar contraseña o rol; límite de intentos de inicio de sesión.
- La evidencia (fotos, clips y video en vivo) **solo se sirve con sesión**; cookie HttpOnly y SameSite=Strict; las acciones que cambian algo exigen la cabecera Authorization.
- **Política de seguridad de contenido** estricta: sin JavaScript en línea ni de CDN.
- **HTTPS** con certificado propio o con Caddy en Docker.
- La contraseña de la cámara **nunca** se escribe en logs ni en la bitácora; el diagnóstico de cámaras solo acepta direcciones de la red local.

9. Privacidad (LFPDPPP)

Los rostros son datos personales sensibles. El sistema está diseñado para guardar lo mínimo y borrarlo solo:

Dato	Se conserva	Por qué
Fotos de eventos sin coincidencia	7 días	Verificar un caso reciente
Eventos sin coincidencia (solo texto)	30 días	Responder "¿pasó tal vehículo?"
Clips de video de alertas	90 días	Evidencia de lo atendido
Alertas y su evidencia	1 año	Historial para auditoría
Rostro de quien no está en la lista negra	Nunca se guarda	Se descarta en memoria

- Dar de alta a una persona exige declarar el **fundamento legal**; queda en la bitácora.
- Los avisos al celular van **sin foto** salvo que el aviso de privacidad lo contemple (en el stand se activa con el letrero de videovigilancia de demostración a la vista).
- La copia de respaldo que sube a la nube **no lleva** contraseñas de cámaras, tokens ni fotos.
- En un despliegue real hace falta un **aviso de privacidad visible** donde haya cámaras y cuidado especial con menores de edad.

10. Validación y resultados

Lectura de placas

Condición	Modelo anterior (YOLOv5 + EasyOCR)	Actual (YOLOv9 + OCR de placas)
De frente	4/5	5/5

De lado 35° / 50°	2/5 · 2/5	5/5 · 5/5
Desde arriba	2/5	5/5
Rotada 15°	3/5	5/5
Lejos	3/5	5/5
Poca luz	4/5	5/5
Tiempo por foto	50–280 ms	22–29 ms

Alcance medido. La placa tiene que medir al menos 36 px de ancho para leerse. De frente, con el stream principal de 3200 px: 10.2 m con lente de 2.8 mm, 15.1 m con 4 mm y 26.6 m con 6 mm. Con el sub-stream de 1280 px: 4.1, 6.0 y 10.6 m. En ángulo de 45° se multiplica por 0.7.

Rendimiento (laptop con RTX 4050, 6 GB)

Escenario	fps por cámara	Total	GPU	VRAM
1 cámara, 4 detectores	13–17	~17	42 %	0.9 GB
4 cámaras en 1 proceso	2.8	~11	34 %	2.0 GB
4 cámaras en 4 procesos	4.6	~18	77 %	3.2 GB

Con 8 fps por cámara, un nodo así atiende **2 cámaras con los cuatro detectores**; una cámara solo de placas o solo de movimiento pesa menos. Con varias cámaras en un mismo proceso la GPU quedaba al 34 %: el límite era Python (un hilo a la vez), no la GPU. **Un proceso por cámara rinde 67 % más**, y así lo hace ahora el catálogo. Datos y método en docs/evidencias/.

Con las dos cámaras reales del stand (7 de octubre)

Cámara	Cuadros analizados por segundo	Placas	Rostros
cam-01 (192.168.100.64)	7.4	38 ms	19 ms
cam-02 (192.168.100.65)	7.7	38 ms	15 ms

Cada cámara en su proceso con placas y rostros, a 8 fps pedidos: la GPU quedó al 38 % con 1.6 de 6 GB y la CPU de todo el equipo al 37 % (muestra de 5 s). Hay margen para activar movimiento y pose.

Respaldo y recuperación

El 7 de octubre se restauró el sistema desde la copia en OneDrive, como si la laptop se hubiera perdido: huellas SHA-256 correctas, el código en el mismo commit, la base de datos íntegra y con los mismos registros (236 eventos, 24 alertas), y las pruebas pasando sobre el código restaurado. Restaurar tomó 2 segundos más la instalación de dependencias.

Calidad del software

- **277 pruebas automáticas** en 24 archivos, sin cámara ni GPU. La integración continua las corre en Windows y Linux (Python 3.11 y 3.12) y contra PostgreSQL 16 y Redis 7, y construye la imagen Docker.

- Las pruebas incluyen una cámara simulada (ISAPI con Digest y RTSP) que verifica que con una contraseña mala se hace **un solo intento**.
- Operación registrada con las cámaras Hikvision (septiembre–octubre): 25 lecturas de placa, 12 coincidencias con la lista negra, 20 alertas y una caída de cámara recuperada en 30 s.

11. Instalación y uso

Requisitos

- Windows 10/11 o Linux, Python 3.11 o superior.
- GPU NVIDIA (probado con RTX 4050 de 6 GB). Sin GPU funciona en CPU, de 10 a 20 veces más lento.
- Cámaras IP con RTSP en la misma red (probado con Hikvision DS-2CD1063G2-LIU de 6 MP).

Instalación

```
python -m venv venv
venv\Scripts\activate
pip install torch==2.13.0+cu126 torchvision==0.28.0+cu126 --index-url
https://download.pytorch.org/whl/cu126
pip install -r requirements.txt -c constraints.txt
copy .env.example .env
python tools/init_plataforma.py      (crea la base de datos y el usuario administrador)
```

Uso

- **Arrancar:** iniciar_api.bat y abrir http://localhost:8000.
- **Dar de alta una cámara:** apartado Cámaras → Buscar → Diagnosticar → Recomendación → Dar de alta → Iniciar worker.
- **Monitoreo:** video en vivo con las detecciones dibujadas y la lista de detecciones.
- **Lista negra:** placas (con vigencia) y personas (con fundamento legal).
- **Zonas:** icono de zona en el recuadro de cada cámara; se dibuja la zona o la línea y su horario.
- **Registro:** búsqueda por placa, tipo, cámara, fechas o descripción; atender alertas; reporte CSV.
- **Avisos al celular:** NOTIFY_TELEGRAM_TOKEN y NOTIFY_TELEGRAM_CHATS en el .env; se prueban en Administración → Notificaciones → Enviar prueba.
- **Nombre del sitio** en la pantalla de acceso: SITIO_NOMBRE en el .env.
- **Respaldo:** python tools/respaldo.py (local) y --nube (copia sin credenciales a OneDrive).
- **Recuperar en otro equipo:** git clone repo.bundle, instalar, copiar la base de datos del respaldo y volver a escribir el .env (las credenciales nunca viajan en el respaldo de la nube).

12. Limitaciones

- **Armas:** apagado. El modelo general (COCO) no detectó un cuchillo real; hace falta entrenar uno propio.
- **Rostros:** el umbral es el de fábrica; falta calibrarlo con personas reales y consentimiento.
- **Pose y movimiento:** probados con el modelo, sin un incidente real para validar umbrales.

- **Capacidad:** en una laptop como la del equipo, 2 cámaras con todo activo; para más, más nodos.
- **Placas de papel:** el detector está entrenado con placas reales; una placa impresa se detecta mejor sobre un vehículo y de cerca.
- **Licencias de modelos** para uso comercial (ver siguiente sección).

13. Escalabilidad y costos

El sistema crece agregando nodos de borde: cada nodo con GPU atiende sus cámaras y todos reportan a la misma API, que corre en varios procesos con PostgreSQL y Redis. Se despliega con Docker y HTTPS.

Arquitectura híbrida para varios centros de mando

Dónde	Qué vive ahí	Por qué
Cada sitio	Cámaras, nodo de borde con GPU y copia local de la lista negra	El video y los rostros no salen del edificio; la detección sigue sin internet
Servidor central o nube (Docker)	Lista negra maestra cifrada, API, dashboard y respaldo diario	Todos los centros comparten la misma lista; recuperación ante un accidente
Repositorio	Código y configuración de ejemplo (sin .env ni modelos)	Un equipo nuevo se levanta en minutos; los modelos se descargan de su fuente oficial

No se manda el video a la nube: costaría ancho de banda y una GPU rentada por servidor, y sacaría del sitio datos biométricos. Lo que viaja son eventos y fotos, que pesan poco. Hoy el prototipo corre completo en un equipo; la lista negra maestra compartida es el siguiente paso de arquitectura.

Piloto: un acceso vehicular y uno peatonal	Costo de referencia (MXN)
2 cámaras Hikvision DS-2CD1063G2-LIU (6 MP)	\$4,608
1 nodo con GPU RTX 4050 (laptop o equipo similar)	\$17,000 – \$26,000
Software GOSS IP	Propio
Switch PoE, cableado e instalación	Por cotizar en sitio
Total de equipo	\$21,600 – \$30,600

Precios publicados el 6 de octubre de 2026 en hikvision-store.mx (cámara) y en Mercado Libre, Amazon y Cyberpuerta (laptops con RTX 4050). Para uso comercial: YOLO11 es AGPL-3.0 (publicar el código o licencia Enterprise) y los pesos de rostros buffalo_l son de uso no comercial (alternativa: AuraFace, Apache-2.0).

14. Próximos pasos

- Piloto en un acceso de un campus, con aviso de privacidad y reglas acordadas con el área de seguridad.
- Calibrar el umbral de rostros con personas reales que den su consentimiento.
- Entrenar un modelo propio de armas con el dataset que el sistema ya junta de las alertas calificadas.
- Resolver licencias para uso comercial (detector de personas con licencia Apache o Enterprise; AuraFace para rostros).
- Integración con centros de monitoreo tipo C5 por webhook y avisos en una app móvil.

- Lista negra maestra compartida entre centros de mando, con copia local en cada sitio, y respaldo diario automático.

15. Mejoras incorporadas en la Fase 2

Control de cambios de la rama innovatics-final, tomado del historial de Git (cada fila es un commit verificable). Se actualiza cada día con lo que resulte de las asesorías.

Fecha y hora	Commit	Mejora	Por qué
6 oct 11:31	0130a8d	Arquitectura de red y despliegue para campus; plan de validación de campo	Cómo se instalaría en la universidad
6 oct 20:00	5ad34f8	Respaldo completo con manifiesto SHA-256 (tools/respaldo.py)	Volver a una versión funcional en minutos
6 oct 20:34	7f0afb1	Video local como cámara, en bucle y a velocidad real	Plan B de la demo
6 oct 20:34	67527b4	Catálogo de cámaras: buscar, diagnosticar, recomendar y dar de alta	Instalar sin terminal; escalar a más cámaras
6 oct 20:48	ea5115a	Salvaguarda en la purga de evidencia	Las pruebas llegaron a borrar fotos reales; ya no pueden
6 oct 20:52	fa69b2c	Placa de prueba imprimible, validada con los modelos	Accesorio de respaldo
6 oct 21:08	8493a02	Métricas reales y latido por ventana; medición de capacidad	Un proceso por cámara rinde 67 % más
6 oct 21:10	4e01a08	Reconoce una cámara por serie o MAC si cambia de IP	Le pasó dos veces a la cámara del equipo
6 oct 21:22	c244f4f	Busca cámaras en todas las redes del equipo, también sin internet	La red del stand no tiene internet
6 oct 21:52	d6d2a3d	RTSP DESCRIBE autenticado en la misma conexión	La Hikvision rechazaba la contraseña correcta (probado con la cámara real)
6 oct 23:39	e7897e6	Video en vivo a 20 fps aparte de la IA; revisión de placas; 3 lecturas coincidentes; rostro reportado en cuanto se ve de frente	Video fluido; menos falsos positivos; alerta con la persona presente
7 oct 00:05	—	Avisos por Telegram configurados y probados	La alerta llega al celular
7 oct 00:16	—	Copia en la nube y restauración probada	Respaldo técnico de los lineamientos
7 oct 00:33	265d993	Pantalla de acceso con identidad propia; movimiento en el dashboard	Producto con identidad, no plantilla
7 oct 00:59	9b58285	Loop del logo para el monitor del stand	Identidad del stand
7 oct			
8 oct			
9 oct			